

Java Backend Interview Series

Chapter 1 · Sub-Chapter 1.4

Collections — Map

HashMap · LinkedHashMap · TreeMap · ConcurrentHashMap · EnumMap · WeakHashMap

Baeldung-style Tutorial · 25+ Q&A · Import Blocks · Working Examples · Pitfalls

1 Map Interface — Why Map Doesn't Extend Collection

1.1 Introduction — The Problem

Before Java 2, developers used Hashtable for key-value storage — a synchronized, legacy class with significant limitations. With Java 2's Collections Framework, the Map<K,V> interface arrived, but it was deliberately kept separate from Collection<E>. New developers often ask: why doesn't Map extend Collection? This section explains the design decision and covers all core Map methods.

1.2 Why Map Does NOT Extend Collection

A Collection<E> is a bag of elements — it has add(E e), iterator(), and contains(Object o). A Map<K,V> holds key-value associations — the concept of a "single element" doesn't apply cleanly. If Map extended Collection<Map.Entry<K,V>>, then add(entry) would have to make sense semantically, which conflicts with the put semantics of key replacement. Additionally, methods like size() would be ambiguous — does it count keys or entries? The designers kept Map as a completely separate hierarchy.

TIP: Map is not a Collection, but Map.entrySet(), Map.keySet(), and Map.values() all return Collection views. This bridges the two hierarchies without compromising design.

1.3 Core Map Methods — Before (Verbose) vs After (Modern)

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;

// BEFORE Java 8 – verbose, manual null-checks
Map<String, List<Integer>> scores = new HashMap<>();

if (!scores.containsKey("Alice")) {
    scores.put("Alice", new ArrayList<>());
}

scores.get("Alice").add(95);

// Merge pattern – old style
Integer count = map.get(key);

if (count == null) count = 0;

map.put(key, count + 1);
```

Before Java 8: verbose Map operations

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;

// AFTER Java 8 – concise and atomic
Map<String, List<Integer>> scores = new HashMap<>();
```

```

// computeIfAbsent – creates list lazily, returns it
scores.computeIfAbsent("Alice", k -> new ArrayList<>()).add(95);
scores.computeIfAbsent("Alice", k -> new ArrayList<>()).add(88); // same list

// merge – atomic increment
map.merge(key, 1, Integer::sum); // inserts 1, or adds 1 to existing

// putIfAbsent – only puts if key absent; returns current value
map.putIfAbsent("Bob", 42); // returns null (inserted)
map.putIfAbsent("Bob", 99); // returns 42 (not overwritten)

// getOrDefault – no NPE
int val = map.getOrDefault("Unknown", 0);

// replaceAll – transform all values in-place
map.replaceAll((k, v) -> v * 2);

// forEach – concise iteration
map.forEach((k, v) -> System.out.println(k + "=" + v));

```

After Java 8: concise atomic Map operations

1.4 Iterating a Map — Three Views

```

import java.util.Map;
import java.util.HashMap;

Map<String, Integer> map = Map.of("A", 1, "B", 2, "C", 3);

// 1. entrySet() – most efficient, gives both key and value
for (Map.Entry<String, Integer> e : map.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}

// 2. keySet() – then get() – avoid: double lookup
for (String key : map.keySet()) {
    System.out.println(key + " -> " + map.get(key)); // extra lookup!
}

// 3. values() – no keys
map.values().forEach(System.out::println);

// Stream-based
map.entrySet().stream()
    .filter(e -> e.getValue() > 1)

```

```
.map(Map.Entry::getKey)
.forEach(System.out::println);
```

Three iteration views — prefer `entrySet()` for key+value

WARNING: Never use `keySet()` + `get()` when you need both key and value. Use `entrySet()` to avoid a redundant lookup per entry.

Method	Returns	Notes
<code>put(k,v)</code>	Previous value or null	Replaces existing
<code>get(k)</code>	Value or null	Use <code>getOrDefault</code> to avoid null
<code>getOrDefault(k,def)</code>	Value or def	Null-safe
<code>putIfAbsent(k,v)</code>	Current value or null	Atomic only on <code>ConcurrentHashMap</code>
<code>computeIfAbsent(k,fn)</code>	Current or new value	Lazy init; fn called only if absent
<code>computeIfPresent(k,fn)</code>	New value or null	fn called only if present
<code>compute(k,fn)</code>	New value	fn always called; null result removes key
<code>merge(k,v,fn)</code>	New value	Put v if absent; else <code>fn(old,v)</code>
<code>remove(k)</code>	Removed value or null	
<code>containsKey(k)</code>	boolean	O(1) for <code>HashMap</code>
<code>entrySet()</code>	<code>Set<Entry<K,V>></code>	Live view of map entries
<code>keySet()</code>	<code>Set<K></code>	Live view of keys
<code>values()</code>	<code>Collection<V></code>	Live view; may contain duplicates

2 HashMap Deep Dive — Buckets, Hash, Resize, Treeification

2.1 Introduction

HashMap is the most widely used Map implementation. Understanding its internals — bucket arrays, hash spreading, load factor, resizing, and Java 8's treeification — is critical for interviews and for writing performant applications. A badly designed hashCode() can degrade HashMap from $O(1)$ to $O(n)$ per operation.

2.2 Internal Structure — Node[] Table

HashMap maintains an array of Node<K,V> (called table). Each Node stores: hash (int), key (K), value (V), and next (Node<K,V>). The array is initially null and lazily initialized on first put(). Default initial capacity is 16 (must be power-of-2). The load factor threshold is 0.75 — when size/capacity > 0.75, the table doubles.

```
import java.util.HashMap;
import java.util.Map;

// Simplified Node (actual JDK source)
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next; // chain for collisions
}

// Index formula: which bucket does a key go into?
// hash(key) spreads bits; (n-1) & hash selects bucket
static final int hash(Object key) {
    int h;
    return (key == null) ? 0
        : (h = key.hashCode()) ^ (h >> 16); // XOR upper/lower 16 bits
}

int index = (n - 1) & hash; // n = table.length (power-of-2)
```

HashMap Node structure and hash() spreading function

2.3 Load Factor 0.75 and Resize

The load factor of 0.75 balances time vs. space. At 0.75 occupancy, the probability of long chains is low. When size exceeds capacity * 0.75, HashMap doubles the table (newCap = oldCap << 1) and rehashes all entries. This is $O(n)$ but amortized over many puts. If you know the expected size upfront, use new HashMap<>(expectedSize / 0.75 + 1) to avoid resizes.

```
import java.util.HashMap;
import java.util.Map;

// Avoid repeated resizing when size is known
```

```

int expectedSize = 1000;

Map<String, Integer> map = new HashMap<>(
    (int)(expectedSize / 0.75) + 1 // initial capacity = 1334
);

// Default: new HashMap<>() – capacity 16, load factor 0.75
// Resizes at 12 entries, 24, 48, 96... (doubles each time)

// Custom load factor (rarely needed)

Map<String, Integer> dense = new HashMap<>(16, 0.5f); // resize at 8

```

Pre-sizing HashMap to avoid resize overhead

2.4 Java 8 Treeification — From List to Red-Black Tree

Before Java 8, all collisions in one bucket formed a singly-linked list. With a bad hashCode(), all keys could land in bucket 0 — making get/put O(n). Java 8 introduced treeification: when a single bucket's chain length reaches TREEIFY_THRESHOLD (8) AND the table capacity is >= MIN_TREEIFY_CAPACITY (64), the chain is converted to a TreeNode (Red-Black tree), giving O(log n) worst case. When entries are removed and the tree shrinks to UNTREEIFY_THRESHOLD (6), it reverts to a list.

```

import java.util.HashMap;
import java.util.Map;

// Key constants in HashMap (Java 8+)

static final int TREEIFY_THRESHOLD = 8; // chain -> tree
static final int UNTREEIFY_THRESHOLD = 6; // tree -> chain
static final int MIN_TREEIFY_CAPACITY = 64; // table must be >= 64

// If capacity < 64 but chain >= 8, HashMap RESIZES instead of treeifying
// This is because small tables with long chains benefit more from spreading

// Demonstration: all keys hash to same bucket (worst case)
Map<BadKey, String> map = new HashMap<>();
for (int i = 0; i < 20; i++) map.put(new BadKey(i), "v" + i);
// With BadKey.hashCode() = 0: all in bucket 0
// Java 7: O(n) linked list
// Java 8+: O(log n) Red-Black tree after 8 entries (capacity >= 64)

```

Java 8 treeification thresholds

WARNING: HashMap is NOT thread-safe. Concurrent modifications cause ConcurrentModificationException (fail-fast iterator) or infinite loops in Java 7 (resize race). Use ConcurrentHashMap in multithreaded code.

2.5 computeIfAbsent vs putIfAbsent vs merge

```

import java.util.HashMap;
import java.util.Map;
import java.util.List;

```

```

import java.util.ArrayList;

Map<String, List<String>> groups = new HashMap<>();

// computeIfAbsent: only calls fn if key absent; fn result stored & returned
groups.computeIfAbsent("fruits", k -> new ArrayList<>()).add("apple");
groups.computeIfAbsent("fruits", k -> new ArrayList<>()).add("banana");
// groups = {fruits=[apple, banana]}

// putIfAbsent: stores given value if absent; does NOT return the list
groups.putIfAbsent("veggies", new ArrayList<>()); // returns null
groups.get("veggies").add("carrot"); // separate get needed

// merge: great for counters / accumulators
Map<String, Integer> wordCount = new HashMap<>();
String[] words = {"a", "b", "a", "c", "a", "b"};
for (String w : words) {
    wordCount.merge(w, 1, Integer::sum);
}
// wordCount = {a=3, b=2, c=1}

// compute: always calls fn; null return removes key
wordCount.compute("a", (k, v) -> v == null ? 1 : v + 10);
// a -> 13
wordCount.compute("a", (k, v) -> null); // removes "a"

```

computeIfAbsent vs putIfAbsent vs merge — use case comparison

3 LinkedHashMap — Insertion/Access Order & LRU Cache

3.1 Introduction

LinkedHashMap extends HashMap and adds a doubly-linked list running through all entries in the order they were inserted (or in access order if configured). This adds $O(1)$ overhead per operation but preserves predictable iteration order — unlike HashMap which has undefined order. It is the go-to structure for LRU caches.

3.2 Insertion Order vs Access Order

```
import java.util.LinkedHashMap;
import java.util.Map;

// Default: insertion order
Map<String, Integer> insertion = new LinkedHashMap<>();
insertion.put("C", 3);
insertion.put("A", 1);
insertion.put("B", 2);
System.out.println(insertion); // {C=3, A=1, B=2} – insertion order

// Access order: accessOrder=true – LRU pattern
Map<String, Integer> access = new LinkedHashMap<>(16, 0.75f, true);
access.put("C", 3);
access.put("A", 1);
access.put("B", 2);
access.get("C"); // moves C to the tail (most recently used)
System.out.println(access); // {A=1, B=2, C=3} – C moved to end
```

LinkedHashMap: insertion order vs access order

3.3 Building an LRU Cache with removeEldestEntry()

The key hook for LRU caches is overriding `removeEldestEntry(Map.Entry eldest)`. This method is called after every `put()` and `addEntry()`. If it returns `true`, the eldest (least-recently-used) entry is automatically removed. Combined with `accessOrder=true`, this gives a fully working bounded LRU cache in ~10 lines.

```
import java.util.LinkedHashMap;
import java.util.Map;

// Bounded LRU Cache – classic LinkedHashMap pattern
public class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxSize;

    public LruCache(int maxSize) {
        super(maxSize, 0.75f, true); // accessOrder = true
        this.maxSize = maxSize;
    }
}
```



```

}

@Override
protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
    return size() > maxSize; // evict when over capacity
}
}

// Usage
LruCache<Integer, String> cache = new LruCache<>(3);
cache.put(1, "one"); // {1=one}
cache.put(2, "two"); // {1=one, 2=two}
cache.put(3, "three"); // {1=one, 2=two, 3=three}
cache.get(1); // access 1 -> moved to tail
cache.put(4, "four"); // evicts 2 (LRU)
System.out.println(cache); // {3=three, 1=one, 4=four}

```

LRU Cache implemented with `LinkedHashMap.removeEldestEntry()`

TIP: `LinkedHashMap` LRU works for single-threaded use. For thread-safe LRU, wrap with `Collections.synchronizedMap()` or use Caffeine cache library which is far more capable.

WARNING: In access-order mode, `get()` modifies the structure (moves entry to tail). This makes `LinkedHashMap` non-thread-safe even for read-only operations. Always synchronize externally.

4 TreeMap — Sorted NavigableMap with Red-Black Tree

4.1 Introduction

TreeMap implements `NavigableMap<K,V>` (which extends `SortedMap<K,V>`) using a Red-Black tree internally. All operations (put, get, remove) run in $O(\log n)$. Unlike `HashMap`, `TreeMap` keeps keys in their natural sorted order (or by a provided `Comparator`). This makes `TreeMap` the right choice when you need sorted iteration, range queries, or floor/ceiling lookups.

4.2 Natural Order and Custom Comparator

```
import java.util.TreeMap;
import java.util.Comparator;
import java.util.NavigableMap;

// Natural order: String keys sorted lexicographically
TreeMap<String, Integer> natural = new TreeMap<>();

natural.put("banana", 2);
natural.put("apple", 1);
natural.put("cherry", 3);

System.out.println(natural); // {apple=1, banana=2, cherry=3}

// Custom comparator: reverse order
TreeMap<String, Integer> reversed = new TreeMap<>(Comparator.reverseOrder());

reversed.putAll(natural);

System.out.println(reversed); // {cherry=3, banana=2, apple=1}

// Case-insensitive
TreeMap<String, Integer> ci = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);

ci.put("Apple", 10);
ci.put("apple", 20); // same key – overwrites

System.out.println(ci); // {Apple=20}
```

TreeMap with natural and custom Comparator

4.3 NavigableMap Methods — floor, ceiling, headMap, tailMap, subMap

```
import java.util.TreeMap;
import java.util.NavigableMap;

TreeMap<Integer, String> scores = new TreeMap<>();

scores.put(50, "C"); scores.put(70, "B"); scores.put(90, "A");
scores.put(55, "C+"); scores.put(75, "B+");

// floor: greatest key <= 72
System.out.println(scores.floorKey(72)); // 70
```

```

System.out.println(scores.floorEntry(72)); // 70=B

// ceiling: smallest key >= 72
System.out.println(scores.ceilingKey(72)); // 75

// lower/higher: strictly less/greater
System.out.println(scores.lowerKey(70)); // 55
System.out.println(scores.higherKey(70)); // 75

// headMap: all keys strictly < 70
NavigableMap<Integer,String> below70 = scores.headMap(70);
System.out.println(below70); // {50=C, 55=C+}

// tailMap: all keys >= 75
NavigableMap<Integer,String> from75 = scores.tailMap(75);
System.out.println(from75); // {75=B+, 90=A}

// subMap: keys in range [55, 75)
NavigableMap<Integer,String> mid = scores.subMap(55, true, 75, false);
System.out.println(mid); // {55=C+, 70=B}

// descendingMap: reverse-order view (cheap – O(1))
System.out.println(scores.descendingMap()); // {90=A, 75=B+, 70=B, ...}

```

TreeMap NavigableMap methods — floor, ceiling, subMap

WARNING: `headMap()`, `tailMap()`, and `subMap()` return live views backed by the original `TreeMap`. Modifications to the view are reflected in the original and vice versa. They also throw `IllegalArgumentException` if you try to insert a key outside the view's range.

TIP: `TreeMap` guarantees sorted order but costs $O(\log n)$ per operation vs $O(1)$ average for `HashMap`. Use `TreeMap` only when sorted iteration or range queries are needed.

4.4 Common Interview Patterns

```

import java.util.TreeMap;
import java.util.Map;
import java.util.NavigableMap;

// Find all employees with salary in [60000, 90000]
TreeMap<Integer, String> salaries = new TreeMap<>();
salaries.put(50000, "Alice"); salaries.put(75000, "Bob");
salaries.put(90000, "Carol"); salaries.put(120000, "Dave");

salaries.subMap(60000, true, 90000, true)
    .forEach((sal, name) -> System.out.println(name + ": " + sal));

// Bob: 75000

```

```
// Carol: 90000

// Priority-based event scheduling: get next event <= now
TreeMap<Long, Runnable> events = new TreeMap<>();

long now = System.currentTimeMillis();

Map.Entry<Long, Runnable> next = events.floorEntry(now);

if (next != null) next.getValue().run();
```

TreeMap range queries and scheduling patterns

5 ConcurrentHashMap — Thread-Safe Map with CAS

5.1 Introduction

ConcurrentHashMap is the thread-safe alternative to HashMap. In Java 7, it used 16 independent Segment locks (striped locking). Java 8 completely redesigned it: the array of Node<K,V> is the same structure as HashMap, but individual bucket heads are locked with synchronized + CAS (Compare-And-Swap) operations — far more scalable than a single lock.

5.2 Java 7 vs Java 8 Locking Model

Aspect	Java 7 (Segment)	Java 8 (CAS + synchronized)
Lock granularity	16 segments (default)	Per bucket (much finer)
Read	Volatile, no lock	Volatile, no lock
Write	Lock on segment	CAS for empty, synchronized head for collision
Resize	Per-segment	Cooperative — multiple threads help transfer
Null key/value	Not allowed	Not allowed
size()	Sum segment counts	sumCount() — approximate for large maps

5.3 Why No Null Keys or Values

HashMap allows one null key and null values. ConcurrentHashMap explicitly disallows both. The reason: in a concurrent context, if get(key) returns null, you cannot distinguish between "key not present" and "key maps to null". This ambiguity would require a separate containsKey() call — creating a TOCTOU race condition. By banning null entirely, ConcurrentHashMap makes every null return unambiguous: key is absent.

WARNING: ConcurrentHashMap forbids null keys AND null values. Attempting to put(null, ...) or put(key, null) throws NullPointerException. Design your domain to use Optional<V> or sentinel values if null semantics are needed.

5.4 Atomic Compute Operations

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.List;

// Thread-safe word count using ConcurrentHashMap
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

// merge – atomic: if absent put 1, else add 1
String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};
for (String w : words) {
    counts.merge(w, 1, Integer::sum);
}

// {apple=3, banana=2, cherry=1}

// computeIfAbsent – safe lazy initialization
```

```

ConcurrentHashMap<String, List<String>> groups = new ConcurrentHashMap<>();

groups.computeIfAbsent("fruits", k -> new CopyOnWriteArrayList<>()).add("apple");
groups.computeIfAbsent("fruits", k -> new CopyOnWriteArrayList<>()).add("banana");

// compute – atomic read-modify-write
counts.compute("apple", (k, v) -> v == null ? 1 : v + 100);

// Parallel forEach (uses ForkJoinPool.commonPool())
counts.forEach(2, // parallelismThreshold
(k, v) -> System.out.println(Thread.currentThread().getName()
+ ": " + k + "=" + v));

```

ConcurrentHashMap atomic operations and parallel forEach

5.5 putIfAbsent vs computeIfAbsent in Concurrent Code

```

import java.util.concurrent.ConcurrentHashMap;
import java.util.ArrayList;
import java.util.List;

// putIfAbsent: MAY create the value even if not stored
// (because two threads can both enter and create the list)
ConcurrentHashMap<String, List<String>> map = new ConcurrentHashMap<>();
map.putIfAbsent("key", new ArrayList<>()); // WASTED allocation possible

// computeIfAbsent: fn is called AT MOST ONCE per absent key
// (the JDK guarantees only one thread's fn result is used)
map.computeIfAbsent("key", k -> new ArrayList<>());

// Safe, no wasted allocations (in most JDK implementations)

// For bulk stats:
long sum = counts.reduceValues(1, Long::sum); // parallel sum
String keys = counts.reduceKeys(1,
k -> k.toUpperCase(),
(a, b) -> a + ", " + b);

```

putIfAbsent vs computeIfAbsent in concurrent context

TIP: Use `ConcurrentHashMap.compute()` for atomic read-modify-write. The entire lambda executes while holding the lock on that bucket — but keep lambdas short to avoid contention.

6 EnumMap — Ordinal-Array-Backed O(1) Map

6.1 Introduction

EnumMap is a specialized Map implementation for enum keys. Internally it uses a simple `Object[]` array indexed by the enum's ordinal (0, 1, 2, ...). There is no hashing, no collision handling, no tree. Every operation is $O(1)$ with extremely low overhead — making EnumMap significantly faster than HashMap when all keys are from the same enum type.

6.2 EnumMap Internals and Usage

```
import java.util.EnumMap;
import java.util.Map;
import java.util.HashMap;

enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

// EnumMap – backed by Object[] of size Day.values().length (7)
EnumMap<Day, String> schedule = new EnumMap<>(Day.class);

schedule.put(Day.MON, "Team standup at 9am");
schedule.put(Day.WED, "Architecture review");
schedule.put(Day.FRI, "Sprint demo");

// get(Day.MON) => array[0], get(Day.WED) => array[2]
System.out.println(schedule.get(Day.MON)); // Team standup at 9am

// Iteration in enum declaration order
schedule.forEach((day, task) -> System.out.println(day + ": " + task));

// MON: Team standup...
// WED: Architecture review...
// FRI: Sprint demo...

// Convert from HashMap to EnumMap for performance boost
Map<Day, Integer> hoursMap = new HashMap<>();
hoursMap.put(Day.MON, 8); hoursMap.put(Day.SAT, 4);

EnumMap<Day, Integer> hours = new EnumMap<>(hoursMap); // copy constructor
```

EnumMap: ordinal-indexed, enum-declaration order iteration

Feature	EnumMap	HashMap with Enum Keys
Backing structure	<code>Object[]</code> by ordinal	Hash table (<code>Node[]</code>)
get/put complexity	$O(1)$ — array index	$O(1)$ average — hash lookup
Memory	Minimal (one array)	Node objects + array
Iteration order	Enum declaration order	Undefined

Feature	EnumMap	HashMap with Enum Keys
Null keys	Not allowed	One null allowed
Performance	Faster (no hashing)	Slightly slower

WARNING: EnumMap only accepts enum type keys. Passing a null key throws NullPointerException. All keys must be from the same enum type specified in the constructor.

7 WeakHashMap — GC-able Keys for Metadata Caches

7.1 Introduction

WeakHashMap uses WeakReference for its keys. When a key has no other strong references and the GC runs, the key-value entry is automatically removed from the map. This makes WeakHashMap ideal for metadata caches where the value should live exactly as long as the key object — no manual cleanup needed.

7.2 Weak References and Entry Expiry

```
import java.util.WeakHashMap;

WeakHashMap<Object, String> metadata = new WeakHashMap<>();

Object key1 = new Object();
Object key2 = new Object();

metadata.put(key1, "metadata for key1");
metadata.put(key2, "metadata for key2");

System.out.println("Size before GC: " + metadata.size()); // 2

key1 = null; // Remove strong reference to key1
System.gc(); // Hint to GC (not guaranteed)
Thread.sleep(100); // Give GC time to run

// Entry for key1 may have been evicted
System.out.println("Size after GC: " + metadata.size()); // likely 1
// key2's entry still alive because key2 variable holds strong ref

// Common use: per-object cache (e.g., in frameworks)
WeakHashMap<Class<?>, TypeDescriptor> typeCache = new WeakHashMap<>();
typeCache.put(String.class, new TypeDescriptor("String", 0));
```

WeakHashMap: entries auto-removed when keys become weakly reachable

7.3 When to Use WeakHashMap

Use Case	Why WeakHashMap is Appropriate
Framework metadata caches (e.g., Spring BeanFactory)	Cache should expire when the bean class is unloaded
Per-object augmentation (attach data to instances)	Data should disappear when the object is GC'd
ClassLoader-keyed caches	Prevents ClassLoader leaks in app servers
Listener registries where caller manages lifetime	Listeners auto-deregistered when GC'd

WARNING: WeakHashMap is NOT thread-safe. Use `Collections.synchronizedMap(new WeakHashMap<>())` for concurrent access. Also, WeakHashMap values are strongly referenced — if a value holds a reference back to its key, the key is never GC'd (retention cycle).

TIP: WeakHashMap is different from SoftHashMap (no built-in SoftHashMap in JDK). For memory-sensitive caches, consider Guava's CacheBuilder with softValues() or weakKeys() which provides proper eviction with concurrency support.

8 Choosing the Right Map — Decision Table

8.1 Introduction

Selecting the wrong Map type is a common source of bugs (thread-safety), performance issues ($O(\log n)$ when $O(1)$ suffices), and memory leaks (missing WeakHashMap). Use this decision guide to pick the right implementation.

Requirement	Best Choice	Reason
General purpose, single thread	HashMap	$O(1)$ avg, flexible
Preserve insertion order	LinkedHashMap	Maintains doubly-linked list
LRU/MRU cache	LinkedHashMap (accessOrder=true)	removeEldestEntry hook
Sorted by key / range queries	TreeMap	Red-Black tree, NavigableMap API
Thread-safe, high concurrency	ConcurrentHashMap	CAS + fine-grained locking
All keys are enum constants	EnumMap	Ordinal array, fastest possible
Keys should be GC-able	WeakHashMap	Weak references, auto-eviction
Reference equality keys	IdentityHashMap	Uses == not equals()
Immutable, read-only	Map.of() / Map.copyOf()	Compact, null-hostile, thread-safe

8.2 Performance Summary

Map	get/put	Ordering	Thread Safe	Null Keys	Null Values
HashMap	$O(1)$ avg	None	No	1 allowed	Yes
LinkedHashMap	$O(1)$ avg	Insertion/Access	No	1 allowed	Yes
TreeMap	$O(\log n)$	Sorted	No	No	Yes
ConcurrentHashMap	$O(1)$ avg	None	Yes (CAS)	No	No
EnumMap	$O(1)$	Enum ordinal	No	No	Yes
WeakHashMap	$O(1)$ avg	None	No	1 (weak ref)	Yes
IdentityHashMap	$O(1)$ avg	None	No	Yes (==)	Yes

9 Map + Streams — Grouping, Partitioning & Collectors

9.1 Introduction

Java 8 Streams work seamlessly with Maps via Collectors. `Collectors.groupingBy()` produces a `Map<K, List<V>>`. `Collectors.toMap()` creates any Map from a stream. `Collectors.counting()`, `summingInt()`, and `joining()` serve as downstream collectors. Understanding how streams interact with Map internals is a common interview topic.

9.2 `Collectors.groupingBy()` — The Most Useful Collector

```
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;

record Employee(String name, String dept, int salary) {}

List<Employee> employees = List.of(
    new Employee("Alice", "Engineering", 90000),
    new Employee("Bob", "Engineering", 85000),
    new Employee("Carol", "Marketing", 75000),
    new Employee("Dave", "Marketing", 70000),
    new Employee("Eve", "Engineering", 95000)
);

// Group by department -> List of employees
Map<String, List<Employee>> byDept =
    employees.stream()
        .collect(Collectors.groupingBy(Employee::dept));
// {Engineering=[Alice,Bob,Eve], Marketing=[Carol,Dave]}

// Group by dept -> count per dept
Map<String, Long> countByDept =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::dept, Collectors.counting()));
// {Engineering=3, Marketing=2}

// Group by dept -> average salary
Map<String, Double> avgSalaryByDept =
    employees.stream()
        .collect(Collectors.groupingBy(
```

```

Employee::dept,
Collectors.averagingInt(Employee::salary)));

// Ordered result (TreeMap)
Map<String, Long> sortedCounts =
employees.stream()
.collect(Collectors.groupingBy(
Employee::dept, TreeMap::new, Collectors.counting()));
// TreeMap: {Engineering=3, Marketing=2} – alphabetical order

```

Collectors.groupingBy() with downstream collectors

9.3 Collectors.toMap() — Pitfalls and Merge Functions

```

import java.util.List;
import java.util.Map;
import java.util.LinkedHashMap;
import java.util.stream.Collectors;

// Basic toMap: Employee -> name:salary
Map<String, Integer> salaryMap =
employees.stream()
.collect(Collectors.toMap(
Employee::name,
Employee::salary));

// PITFALL: duplicate keys throw IllegalStateException!
List<Employee> withDupe = List.of(
new Employee("Alice", "Engineering", 90000),
new Employee("Alice", "Marketing", 75000) // duplicate name!
);
withDupe.stream().collect(Collectors.toMap(
Employee::name, Employee::salary)); // IllegalStateException!

// FIX: provide merge function to handle duplicates
Map<String, Integer> safeMap =
withDupe.stream()
.collect(Collectors.toMap(
Employee::name,
Employee::salary,
(existing, replacement) -> Math.max(existing, replacement)));
// Alice -> 90000 (max salary wins)

```

```
// toMap with specific Map type (LinkedHashMap for order)
Map<String, Integer> ordered =
employees.stream()
.collect(Collectors.toMap(
Employee::name, Employee::salary,
(a, b) -> a, // keep first on duplicate
LinkedHashMap::new)); // maintain insertion order
```

Collectors.toMap() — duplicate key pitfall and merge function

9.4 partitioningBy() and Counting Patterns

```
import java.util.List;
import java.util.Map;
import java.util.Arrays;
import java.util.stream.Collectors;

// partitioningBy: splits into Map<Boolean, List<T>>
Map<Boolean, List<Employee>> highVsLow =
employees.stream()
.collect(Collectors.partitioningBy(
e -> e.salary() > 80000));
// {true=[Alice,Bob,Eve], false=[Carol,Dave]}

// Frequency map (word count)
String text = "the quick brown fox jumps over the lazy dog the";
Map<String, Long> wordCount =
Arrays.stream(text.split("\\s+"))
.collect(Collectors.groupingBy(
w -> w, Collectors.counting()));
// {the=3, quick=1, brown=1, fox=1, ...}

// Top N by value
wordCount.entrySet().stream()
.sorted(Map.Entry.<String,Long>comparingByValue().reversed())
.limit(3)
.forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
// the: 3 | quick: 1 | ...
```

partitioningBy() and frequency maps

TIP: `Collectors.groupingBy()` defaults to `HashMap`. For sorted results pass `TreeMap::new` as the `mapFactory`. For insertion-order results pass `LinkedHashMap::new`.

11 Map Iteration Patterns & Modification During Iteration

11.1 Introduction — ConcurrentModificationException

HashMap's iterator is fail-fast: it tracks a modCount counter. Every structural modification (put, remove) increments modCount. If the iterator detects modCount changed since it was created, it throws ConcurrentModificationException. This is NOT a thread-safety guarantee — it is a best-effort bug detector for single-threaded misuse.

11.2 Safe Removal During Iteration — Iterator.remove()

```
import java.util.HashMap;
import java.util.Map;
import java.util.Iterator;
import java.util.List;
import java.util.stream.Collectors;

// WRONG: modifying map while iterating with for-each
Map<String, Integer> map = new HashMap<>();
map.put("a", 1); map.put("b", 2); map.put("c", 3);
for (String key : map.keySet()) {
    if (map.get(key) < 2) {
        map.remove(key); // ConcurrentModificationException!
    }
}

// CORRECT: use Iterator.remove() which does not increment modCount
Iterator<Map.Entry<String,Integer>> it = map.entrySet().iterator();
while (it.hasNext()) {
    Map.Entry<String,Integer> e = it.next();
    if (e.getValue() < 2) {
        it.remove(); // safe: uses iterator's internal state
    }
}

// CORRECT Java 8+: removeIf via entrySet
map.entrySet().removeIf(e -> e.getValue() < 2);

// CORRECT: collect keys to remove, then remove after iteration
List<String> toRemove = map.entrySet().stream()
    .filter(e -> e.getValue() < 2)
    .map(Map.Entry::getKey)
```

```
.collect(Collectors.toList());

toRemove.forEach(map::remove);
```

Safe map modification during iteration — four correct patterns

11.3 Sorting a Map by Value

```
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.TreeMap;
import java.util.Map;
import java.util.stream.Collectors;

Map<String, Integer> scores = new HashMap<>();

scores.put("Alice", 95); scores.put("Bob", 82);
scores.put("Carol", 91); scores.put("Dave", 78);

// Sort by value descending – collect to LinkedHashMap (preserves order)
Map<String, Integer> sortedByValue = scores.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new));

sortedByValue.forEach((k, v) ->
    System.out.println(k + ": " + v));

// Alice: 95
// Carol: 91
// Bob: 82
// Dave: 78

// Sort by key – just use TreeMap
Map<String, Integer> sortedByKey = new TreeMap<>(scores);

// Alice: 95, Bob: 82, Carol: 91, Dave: 78 (alphabetical)
```

Sort map by value (LinkedHashMap) and by key (TreeMap)

11.4 Merging Two Maps

```
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.Stream;
```



```

Map<String, Integer> mapA = new HashMap<>(Map.of("a",1,"b",2,"c",3));
Map<String, Integer> mapB = new HashMap<>(Map.of("b",20,"c",30,"d",4));

// Strategy 1: putAll – mapB overwrites mapA on conflict
Map<String, Integer> merged1 = new HashMap<>(mapA);
merged1.putAll(mapB);
// {a=1, b=20, c=30, d=4} – b and c from mapB

// Strategy 2: merge() – custom resolution (sum values)
Map<String, Integer> merged2 = new HashMap<>(mapA);
mapB.forEach((k, v) -> merged2.merge(k, v, Integer::sum));
// {a=1, b=22, c=33, d=4} – b=2+20, c=3+30

// Strategy 3: Stream.concat + toMap with merge function
Map<String, Integer> merged3 =
Stream.concat(mapA.entrySet().stream(), mapB.entrySet().stream())
.collect(Collectors.toMap(
Map.Entry::getKey,
Map.Entry::getValue,
Integer::sum)); // sum on collision

```

Merging two Maps — putAll, merge(), and Stream.toMap()

TIP: Use `merge()` for accumulation (e.g., combining frequency maps). Use `putAll()` when second map wins on conflict. Use `Stream.toMap()` with a merge function for functional pipelines.

WARNING: `Map.putAll()` is NOT atomic. In concurrent code, another thread can observe the map in a partially-merged state. For thread-safe bulk merge, iterate `mapB` and call `ConcurrentHashMap.merge()` per entry.

12 Advanced Interview Deep Dive — Tricky Scenarios

12.1 Scenario: String vs new String() as HashMap Key

```
import java.util.HashMap;
import java.util.Map;
import java.util.IdentityHashMap;

// String literals are interned; new String() creates new object
// But HashMap uses equals() – so BOTH work correctly as keys

Map<String, Integer> map = new HashMap<>();

map.put("hello", 1); // interned literal
map.put(new String("hello"), 2); // new object, same value

System.out.println(map.size()); // 1 – equals() matches
System.out.println(map.get("hello")); // 2 (overwritten)

// With IdentityHashMap (uses ==) – different!
Map<String, Integer> idMap = new IdentityHashMap<>();

String a = "hello"; // interned
String b = new String("hello"); // different reference

idMap.put(a, 1);
idMap.put(b, 2);

System.out.println(idMap.size()); // 2 (different references)

// Practical tip: String.intern() to save memory for repeated strings
String s1 = new String("hello").intern(); // returns pooled instance
String s2 = new String("hello").intern();

System.out.println(s1 == s2); // true (same pooled object)
```

String vs new String() as Map key — equals() vs == behaviour

12.2 Scenario: EnumMap vs HashMap with Enum Keys — Benchmark

```
import java.util.EnumMap;
import java.util.Map;

// Micro-benchmark insight (JMH results approximation):
//
// HashMap<Day, String> get(): ~12 ns (hash + compare + deref)
// EnumMap<Day, String> get(): ~4 ns (ordinal + array access)
//
// The difference: EnumMap is 3x faster for get() on enum keys.
```

```
enum Status { PENDING, PROCESSING, COMPLETED, FAILED, CANCELLED }

// Correct approach: EnumMap for enum keys
EnumMap<Status, String> descriptions = new EnumMap<>(Status.class);
descriptions.put(Status.PENDING, "Waiting for processing");
descriptions.put(Status.PROCESSING, "Currently being processed");
descriptions.put(Status.COMPLETED, "Successfully completed");
descriptions.put(Status.FAILED, "Failed – see error log");
descriptions.put(Status.CANCELLED, "Cancelled by user");

// O(1) via array[ordinal] – no hash, no comparison
String desc = descriptions.get(Status.FAILED); // "Failed – see error log"

// Iteration is ALWAYS in declaration order (PENDING, PROCESSING, ...)
// Useful for display in defined enum order, not alphabetical
```

EnumMap vs HashMap with enum keys — performance and ordering

12.3 Scenario: WeakHashMap Memory Leak Prevention

```
import java.util.WeakHashMap;
import java.util.HashMap;
import java.util.Map;

// PROBLEM: using strong-reference map for per-object metadata
// causes memory leak – objects never GC'd even when unreachable
static final Map<Widget, WidgetMeta> STRONG = new HashMap<>();

void addWidget(Widget w) {
    STRONG.put(w, new WidgetMeta(w)); // w strongly held by map!
    // Even if all other references to w are gone, STRONG keeps w alive
    // This is a classic memory leak in long-running apps
}

// SOLUTION: WeakHashMap – entries auto-expire with the key
static final Map<Widget, WidgetMeta> WEAK = new WeakHashMap<>();

void addWidgetSafe(Widget w) {
    WEAK.put(w, new WidgetMeta(w));
    // When w becomes unreachable (no other strong refs):
    // GC clears the WeakReference -> entry removed from WEAK
    // No memory leak!
}
```

```
// Verify cleanup happens

Widget temp = new Widget();

WEAK.put(temp, new WidgetMeta(temp));

System.out.println("Before: " + WEAK.size()); // 1

temp = null; // remove strong reference

System.gc();

Thread.sleep(100);

System.out.println("After GC: " + WEAK.size()); // 0 (likely)
```

WeakHashMap prevents per-object metadata memory leaks

12.4 Scenario: ConcurrentHashMap forEach vs entrySet().iterator()

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.Map;
import java.util.Iterator;

ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

map.put("a", 1); map.put("b", 2); map.put("c", 3);

// forEach: weakly consistent – sees modifications made AFTER start
// (may or may not)

map.forEach((k, v) -> {
    System.out.println(k + "=" + v);
    map.put("d", 4); // safe! won't throw ConcurrentModificationException
    // but "d" may or may not appear in THIS iteration
});

// entrySet().iterator(): also weakly consistent for ConcurrentHashMap
// Unlike HashMap which throws CME, CHM iterator never throws

Iterator<Map.Entry<String,Integer>> it = map.entrySet().iterator();

while (it.hasNext()) {
    Map.Entry<String,Integer> e = it.next();
    if (e.getValue() == 1) {
        it.remove(); // safe on ConcurrentHashMap iterator
    }
}

// Parallel forEach (Java 8): uses ForkJoin

map.forEach(1, (k, v) -> processEntry(k, v)); // threshold=1: always parallel
```

ConcurrentHashMap forEach vs iterator — weakly consistent semantics

10 Coding Exercises

10.1 Exercise A — Word Frequency with getOrDefault

Given a string, count the frequency of each word using only `HashMap` and `getOrDefault()`. Then find the top-3 most frequent words.

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.stream.Collectors;

// Exercise A Solution

public static Map<String, Integer> wordFrequency(String text) {
    Map<String, Integer> freq = new HashMap<>();
    for (String word : text.toLowerCase().split("\\W+")) {
        if (!word.isEmpty()) {
            freq.put(word, freq.getOrDefault(word, 0) + 1);
        }
    }
    return freq;
}

public static List<String> top3(Map<String, Integer> freq) {
    return freq.entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
        .limit(3)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());
}
```

Exercise A: word frequency map

10.2 Exercise B — Anagram Grouping

Group a list of strings by their sorted character form (anagram key). Return a `Map<String, List<String>>` where each key is the sorted letters and each value is the group of anagrams.

```
import java.util.HashMap;
import java.util.Map;
import java.util.List;
import java.util.ArrayList;
import java.util.Arrays;

// Exercise B Solution
```

```

public static Map<String, List<String>> groupAnagrams(List<String> words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String word : words) {
        char[] chars = word.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars); // sorted form is the anagram key
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
    }
    return groups;
}

// Test
List<String> words = List.of("eat", "tea", "tan", "ate", "nat", "bat");
Map<String, List<String>> result = groupAnagrams(words);
// {aet=[eat, tea, ate], ant=[tan, nat], abt=[bat]}

```

Exercise B: groupAnagrams using computeIfAbsent

10.3 Exercise C — LRU Cache Implementation

Implement a generic LRU cache with `get()` and `put()` methods. `get()` returns -1 if absent. `put()` evicts the least recently used entry if at capacity.

```

import java.util.LinkedHashMap;
import java.util.Map;

// Exercise C Solution – extends LinkedHashMap
class LRUCache extends LinkedHashMap<Integer, Integer> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder=true
        this.capacity = capacity;
    }

    public int get(int key) {
        return super.getOrDefault(key, -1);
    }

    public void put(int key, int value) {
        super.put(key, value);
    }

    @Override

```

```

protected boolean removeEldestEntry(Map.Entry<Integer,Integer> eldest) {
    return size() > capacity;
}

}

// Test

LRUCache cache = new LRUCache(2);

cache.put(1,1); cache.put(2,2);

cache.get(1); // access 1 -> moves to tail

cache.put(3,3); // evicts 2 (LRU)

System.out.println(cache.get(2)); // -1 (evicted)

System.out.println(cache.get(1)); // 1 (still present)

System.out.println(cache.get(3)); // 3

```

Exercise C: LRU Cache using LinkedHashMap

10.4 Exercise D — Thread-Safe Frequency Counter

Given a stream of strings from multiple producer threads, count frequencies safely using `ConcurrentHashMap` and `merge()`.

```

import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

// Exercise D Solution

ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

ExecutorService executor = Executors.newFixedThreadPool(4);

String[] data = {"apple","banana","apple","cherry","banana","apple"};

for (String item : data) {
    executor.submit(() -> counts.merge(item, 1, Integer::sum));
}

executor.shutdown();

executor.awaitTermination(5, TimeUnit.SECONDS);

counts.forEach((k,v) -> System.out.println(k + "=" + v));

// apple=3, banana=2, cherry=1 (order undefined)

```

Exercise D: concurrent frequency counter with `ConcurrentHashMap.merge()`

Cheat Sheet Quick Reference — Map API & Internals

CS.1 Java 8+ Map API at a Glance

Method	Behaviour	Thread-safe on CHM?
put(k,v)	Stores; returns previous value	Yes (volatile write)
get(k)	Returns value or null	Yes (volatile read)
getOrDefault(k,d)	Returns value or d	Yes
putIfAbsent(k,v)	Stores if absent; returns current	Yes (atomic)
computeIfAbsent(k,fn)	Stores fn result if absent; lazy	Yes (bucket lock)
computeIfPresent(k,fn)	Updates if present; null removes	Yes (bucket lock)
compute(k,fn)	Always calls fn; null removes	Yes (bucket lock)
merge(k,v,fn)	Put v if absent; else fn(old,v)	Yes (bucket lock)
replaceAll(fn)	Updates all values in-place	No (not atomic CHM)
remove(k,v)	Removes only if value matches	Yes (atomic on CHM)
entrySet()	Set view of key-value pairs	Read-only thread-safe
forEach(bi)	Iterates key-value pairs	Weakly consistent on CHM

CS.2 HashMap Internal Constants

Constant	Value	Purpose
DEFAULT_INITIAL_CAPACITY	16	Starting array size (power-of-2)
MAXIMUM_CAPACITY	1<<30	Max array size
DEFAULT_LOAD_FACTOR	0.75f	Resize when size > capacity * 0.75
TREEIFY_THRESHOLD	8	Chain -> Red-Black tree
UNTREEIFY_THRESHOLD	6	Red-Black tree -> chain
MIN_TREEIFY_CAPACITY	64	Table must be >= 64 before treeify

CS.3 When to Use Which Map

Scenario	Use
Default general-purpose map	HashMap
Need insertion-order iteration	LinkedHashMap
LRU/MRU bounded cache	LinkedHashMap(accessOrder=true) + removeEldestEntry
Sorted keys / range queries	TreeMap (NavigableMap API)
Concurrent read-write access	ConcurrentHashMap (never null)
All keys are enum constants	EnumMap (ordinal array, O(1), ordered)
Keys should expire when GC'd	WeakHashMap (WeakReference keys)

Scenario	Use
Reference (==) equality keys	IdentityHashMap
Small immutable map (Java 9+)	Map.of() or Map.ofEntries()
Defensive immutable copy	Map.copyOf() (NOT unmodifiableMap)

Q&A Interview Questions — 25+ Standard & Advanced

Q1-Q10 Standard Questions

Q1. Why doesn't Map extend Collection?

Collection holds individual elements; Map holds key-value associations. The add(E)/iterator() semantics of Collection don't map cleanly to Map's put(K,V)/entrySet() model. The designers deliberately kept them as separate hierarchies.

Q2. What is the default initial capacity and load factor of HashMap?

Initial capacity: 16 (must be power-of-2). Load factor: 0.75. When size exceeds capacity * 0.75, HashMap doubles the table and rehashes.

Q3. What does the hash() function in HashMap do?

hash(key) = key.hashCode() ^ (key.hashCode() >>> 16). This XORs the upper 16 bits into the lower 16 bits, spreading hash bits so that keys differing only in upper bits still land in different buckets.

Q4. What is treeification in Java 8 HashMap?

When a single bucket's chain exceeds TREEIFY_THRESHOLD (8) and table capacity >= 64, the linked list is converted to a Red-Black TreeNode, giving O(log n) worst case. Reverts to list at UNTREEIFY_THRESHOLD (6).

Q5. What is the difference between putIfAbsent and computeIfAbsent?

putIfAbsent evaluates its value argument eagerly (the object is created before calling). computeIfAbsent calls the mapping function lazily only if the key is absent. Use computeIfAbsent for expensive object creation to avoid waste.

Q6. How does LinkedHashMap implement access order?

With accessOrder=true, each get/put moves the entry to the tail of the internal doubly-linked list. The head entry is always the least recently accessed. Overriding removeEldestEntry() implements LRU eviction.

Q7. What is the complexity of TreeMap operations?

O(log n) for put, get, remove, floor, ceiling. O(1) for firstKey/lastKey (maintained in tree root path). headMap/tailMap/subMap are O(1) (lazy views).

Q8. Why does ConcurrentHashMap not allow null keys or values?

In concurrent code, get(key)==null is ambiguous: absent or null value. This would require a separate containsKey() — creating a TOCTOU race. Banning null makes null returns unambiguous (key is absent).

Q9. What is the difference between HashMap and Hashtable?

Hashtable is synchronized on every method (coarse lock) and is a legacy class. HashMap is not synchronized but faster. Hashtable doesn't allow null keys/values. Use ConcurrentHashMap instead of Hashtable for thread safety.

Q10. When should you use EnumMap over HashMap?

When all keys are from the same enum type. EnumMap uses an Object[] array indexed by ordinal — no hashing, no collisions, guaranteed O(1), less memory, and iterates in enum declaration order.

Q11-Q20 Intermediate Questions

Q11. How does HashMap's resize work?

When $\text{size} > \text{capacity} * \text{loadFactor}$, HashMap creates a new array of double the capacity and rehashes all entries. Because capacity is always a power-of-2, an entry either stays in the same bucket or moves to bucket $(\text{old_index} + \text{old_capacity})$. This is an optimization that avoids recomputing hash.

Q12. What happens if two keys have the same hashCode but are not equal?

They land in the same bucket (collision). Java 7: chained in a linked list. Java 8+: chained in a linked list until TREEIFY_THRESHOLD (8) is reached, then the list becomes a Red-Black tree.

Q13. Can you use a mutable object as a HashMap key?

Technically yes, but it is dangerous. If you mutate the key after insertion, its hashCode() changes, and HashMap will never find it in the correct bucket. Always use immutable keys (String, Integer, UUID, final fields).

Q14. What is the difference between headMap, tailMap, and subMap in TreeMap?

headMap(toKey) returns all keys < toKey. tailMap(fromKey) returns all keys >= fromKey. subMap(from, fromInclusive, to, toInclusive) returns a view in the range. All return live views backed by the TreeMap.

Q15. How does ConcurrentHashMap's compute() ensure atomicity?

compute() locks the bucket containing the key (synchronized on the first Node of that chain, or inserts with CAS if bucket is empty). The entire BiFunction executes while the lock is held, ensuring atomic read-modify-write.

Q16. What is the difference between Map.of() and Collections.unmodifiableMap()?

Map.of() creates a new compact immutable map that disallows null keys/values. Collections.unmodifiableMap() wraps an existing map — the underlying map can still be modified through the original reference. Map.of() is truly immutable.

Q17. How does WeakHashMap handle entry expiration?

Keys are stored as WeakReference<K>. The JVM enqueues weak references to a ReferenceQueue when keys become weakly reachable. WeakHashMap polls this queue on each access (get, put, size, etc.) to expunge stale entries.

Q18. What is the difference between compute() and merge() in Map?

compute(k, fn): fn(key, currentValue) always called; null currentValue means absent; null return removes key. merge(k, v, fn): if key absent, inserts v; if present, calls fn(oldValue, v); null return removes key. merge is simpler for accumulation.

Q19. Why should you override both equals() and hashCode() for Map keys?

HashMap uses `hashCode()` to find the bucket and `equals()` to find the exact entry. If only `hashCode()` is overridden, two equal keys may hash to the same bucket but never match (`equals` returns false). If only `equals()` is overridden, equal keys may land in different buckets entirely.

Q20. What is the difference between `ConcurrentHashMap` and `synchronizedMap(HashMap)`?

`synchronizedMap` wraps every method with a single `synchronized(mutex)` lock — only one thread at a time. `ConcurrentHashMap` allows concurrent reads (lock-free), concurrent writes to different buckets (fine-grained locks), and atomic compound operations like `compute()`. `ConcurrentHashMap` is far more scalable.

Q21-Q27 Advanced Questions

Q21. How does `ConcurrentHashMap size()` work in Java 8?

`size()` calls `sumCount()` which sums a `baseCount` long and an array of `CounterCell` entries. `CounterCell` uses `@Contended` (padding) to prevent false sharing between CPU cache lines. The size is an approximation under concurrent modifications — it may not reflect the exact state at any single moment.

Q22. Explain the TOCTOU problem with `putIfAbsent` on `HashMap`.

if `(!map.containsKey(k)) map.put(k, createValue(k))` is not atomic — another thread can insert between `containsKey` and `put`. `putIfAbsent` on `HashMap` is also not atomic. Only `ConcurrentHashMap.putIfAbsent` and `computeIfAbsent` are atomic.

Q23. How does `TreeMap` maintain sorted order after modifications?

`TreeMap` uses a Red-Black tree that self-balances after every insertion and deletion via rotations and recoloring. The invariant (`left subtree < node < right subtree`) is maintained at all times, giving $O(\log n)$ for all operations including `firstKey()` and `lastKey()`.

Q24. What happens when you use a `Comparator` that's inconsistent with `equals()` in `TreeMap`?

`TreeMap` uses the `Comparator` for equality too (`comparator.compare(a,b)==0` means equal, not `a.equals(b)`). If the comparator disagrees with `equals()`, two objects equal by `equals()` may coexist as different keys, violating the Map contract. The `TreeMap`'s behavior is well-defined but surprising.

Q25. How does `LinkedHashMap` maintain insertion order alongside `HashMap O(1)`?

Each `HashMap Node` is extended with `before/after` pointers forming a doubly-linked list. These pointers are maintained in constant time on every `put/remove` — no extra traversal. The linked list head is the eldest entry; tail is the newest. This adds just two references per entry.

Q26. What is `IdentityHashMap` and when should you use it?

`IdentityHashMap` uses `==` (reference equality) and `System.identityHashCode()` instead of `equals()` and `hashCode()`. Use it when you want to track specific object instances regardless of their `equals()` implementation — e.g., object graph traversal, serialization, proxy detection.

Q27. How would you implement a thread-safe bounded cache without Caffeine?

Extend `LinkedHashMap` with `accessOrder=true` and override `removeEldestEntry()`. Wrap with `Collections.synchronizedMap()`. This is a simple LRU cache. For concurrent access without global lock, use `ConcurrentHashMap` with LRU logic tracked separately, or simply use Caffeine/Guava cache.

Summary Chapter Summary & Quick Reference

This chapter covered all six major Map implementations in depth. HashMap provides $O(1)$ average operations using a `Node[]` array, hash spreading, and treeification at 8. LinkedHashMap adds doubly-linked list for insertion/access order and is the foundation for LRU caches. TreeMap uses a Red-Black tree for sorted keys with $O(\log n)$ and NavigableMap range queries. ConcurrentHashMap provides thread-safe operations via CAS and fine-grained bucket locking with no null keys/values. EnumMap uses ordinal-indexed arrays for maximum enum key performance. WeakHashMap auto-evicts entries when keys are GC'd.

TIP: Golden rule: default to HashMap. Upgrade to LinkedHashMap if insertion order matters, TreeMap if sorted order is needed, ConcurrentHashMap for concurrency, EnumMap for enum keys, WeakHashMap for auto-expiring metadata.